

KUPC

Kathmandu University Placement Connect

Phase 2 — Authentication & Authorization

Detailed Implementation Guide — Backend Engineering

Document Control

Field	Value
Project	KUPC — Kathmandu University Placement Connect
Component	Backend — Phase 2: Authentication & Authorization
Version	1.0 (expanded implementation guide)
Status	Draft — ready for implementation
Primary stack	Node.js, Express.js, TypeScript, Supabase Auth, PostgreSQL, JWT, Zod
Depends on	Phase 1 — Project Setup & Architecture (must be complete)
Feeds into	Phase 3 — Student & Company Profiles (consumes req.user + roles)

Table of Contents

1. Phase Goal & Definition of Done
2. Architectural Decision: Feature-Based Modules
3. Milestone 1 — Auth Module Foundation
4. Milestone 2 — Student Registration & OTP
5. Milestone 3 — Company Registration
6. Milestone 4 — Login (Student / Company / Admin)
7. Milestone 5 — Authentication Middleware
8. Milestone 6 — Role-Based Authorization (RBAC)
9. Milestone 7 — Pending Company Middleware
10. Milestone 8 — Refresh Token Rotation
11. Milestone 9 — Logout & Session Revocation
12. Milestone 10 — Protected Endpoints
13. Milestone 11 — Error Handling & Error Codes
14. Milestone 12 — Request Validation (Zod)
15. Milestone 13 — Testing Matrix
16. Data Model Reference
17. Sequence Diagrams (Described)
18. Final Project Structure & Exit Checklist

1. Phase Goal & Definition of Done

Phase 2 turns the empty Express skeleton from Phase 1 into a backend with real identity: students and companies can create accounts, verify who they are, log in, stay logged in safely, and hit protected routes according to their role. Every later phase (profiles, resumes, swiping, chat) depends on `req.user` being reliable, so this phase is treated as a hard dependency gate — nothing in Phase 3 should start until every item below is true.

Definition of Done

- Students can register with a KU email and verify via OTP
- Companies can register and are placed into a pending state
- Admins can log in through a separate, non-discoverable route
- Login issues a short-lived access token and a long-lived, rotating refresh token
- Refresh tokens rotate on every use; reuse of a revoked token kills the whole session family
- Every protected route runs through authentication middleware before reaching a controller
- Roles (STUDENT / COMPANY / ADMIN) are enforced via RBAC middleware, returning 403 on mismatch
- Companies with `verification_status = pending` are blocked from restricted actions (e.g. posting jobs) with a specific `PENDING_VERIFICATION` code
- Logout revokes the refresh token and deletes the session row — it does not just discard the client-side JWT
- All auth endpoints are validated with Zod and covered by the test matrix in Milestone 13

2. Architectural Decision: Feature-Based Modules

The Phase 1 specification used a layer-based layout (top-level controllers/, services/, routes/). That structure is fine for a thin skeleton, but authentication is the first feature-rich module in KUPC, and swiping, chat, and resume analysis will follow the same shape. From Phase 2 onward, the codebase moves to a **feature-based** layout: everything that belongs to one domain — routes, controller, service, validation, types — lives together under `modules/<feature>/`. This keeps related code co-located, makes it obvious where to add new auth logic, and lets a module be extracted into its own service later with minimal disruption.

Module responsibilities

File	Responsibility
<code>auth.routes.ts</code>	Declares Express routes, wires middleware, delegates to controller
<code>auth.controller.ts</code>	Thin HTTP layer: parses request, calls service, shapes response envelope
<code>auth.service.ts</code>	All business logic: Supabase calls, token issuance, OTP checks, DB writes
<code>auth.validation.ts</code>	Zod schemas for every auth endpoint's body/query/params
<code>auth.types.ts</code>	TypeScript interfaces/DTOs shared across the module
<code>auth.constants.ts</code>	Error codes, OTP length/expiry, token TTLs, role enum
<code>index.ts</code>	Re-exports the router so <code>app.ts</code> only imports one thing per module

3. Milestone 1 — Auth Module Foundation

3.1 Folder Structure

```
src/  
  modules/auth/ - auth.controller.ts · auth.service.ts · auth.routes.ts · auth.validation.ts ·  
  auth.types.ts · auth.constants.ts · index.ts  
  middleware/ - authenticate.ts · authorize.ts · requireVerifiedCompany.ts · attachUser.ts  
  config/ - supabase.ts · env.ts  
  database/ - users.repository.ts · sessions.repository.ts · refreshTokens.repository.ts  
  types/ - express.d.ts  
  utils/ - jwt.ts · auth.ts (password/OTP helpers)
```

3.2 Supabase Client Configuration

Two Supabase clients are created, never one: an anon client used for anything performed on behalf of an end user (sign-up, sign-in, OTP), and a service_role client used only in trusted server-side code for privileged operations (creating rows across RLS boundaries, admin actions). The service-role key must never be sent to a client or logged.

```
// config/supabase.ts  
export const supabaseAnon = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);  
export const supabaseAdmin = createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, {  
  auth: { persistSession: false }  
});
```

Verify the connection with a trivial query (e.g. `select now()`) at boot time and fail fast with a clear log line if the environment variables are wrong — do not implement login logic in this step.

3.3 Environment Variables

```
SUPABASE_URL=  
SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
JWT_SECRET=  
JWT_EXPIRES_IN=15m  
REFRESH_EXPIRES_IN=30d  
OTP_LENGTH=6  
OTP_EXPIRES_IN=10m  
OTP_MAX_ATTEMPTS=5  
KU_EMAIL_DOMAIN=ku.edu.np
```

All variables are loaded and validated once at startup through a small Zod-checked `config/env.ts` module, so a missing or malformed variable crashes the process immediately instead of surfacing as a confusing runtime error later.

4. Milestone 2 — Student Registration & OTP

4.1 Registration Validation

RegisterStudentSchema (Zod): email (must match *.ku.edu.np pattern, otherwise reject with INVALID_EMAIL_DOMAIN), full_name (2–100 chars), password (min 8 chars, at least one number — Supabase Auth enforces its own minimum as well).

4.2 Registration Endpoint

POST /api/v1/auth/register/student *Public*

Validates KU email, creates a Supabase Auth user, creates a users row (role=STUDENT, email_verified=false), sends OTP.

Request: { "email": "user@ku.edu.np", "full_name": "...", "password": "..." }

Response: { "success": true, "data": { "otp_sent": true, "expires_in": 600 } }

Flow: validate body → check email domain → Supabase Auth signUp() → insert users row inside the same transaction as the students row (status = unverified) → trigger OTP send → return success without issuing any token yet. No token is issued until OTP verification succeeds — an unverified account must not be able to authenticate.

4.3 OTP Verification

POST /api/v1/auth/verify-otp *Public*

Verifies the 6-digit OTP, marks the user verified, issues access + refresh tokens.

Request: { "email": "user@ku.edu.np", "otp": "482913" }

Response: { "data": { "access_token", "refresh_token", "user": {...} } }

- OTP expires after 10 minutes (configurable) and allows a maximum of 5 attempts before the code is invalidated and a new one must be requested
- On success: email_verified = true, a session row is created, a refresh token is generated and stored hashed, and an access JWT (15 min TTL) is returned
- On failure: return INVALID_OTP or OTP_EXPIRED; never reveal whether the email itself exists in error messages, to avoid account enumeration

4.4 Store User Profile

After successful verification, the service layer writes the permanent identity rows in one transaction: users (id, email, role=STUDENT, email_verified=true, status=active) and students (id fk, ku_id, full_name, graduation_year=null, department=null — filled in during Phase 3 profile setup).

5. Milestone 3 — Company Registration

5.1 Company Validation

CompanyRegisterSchema: company_name (2–150 chars), email (valid format, uniqueness checked against users table), password (same policy as students), website (valid URL, optional at registration but required before approval in Phase 11's verification workflow).

5.2 Company Signup

POST /api/v1/auth/register/company *Public*

Creates a Supabase Auth user and a companies row with verification_status = pending. No OTP step — company email is confirmed via a standard Supabase confirmation link instead, since companies are not restricted to a KU domain.

Request: { "company_name", "email", "password", "website" }

Response: { "data": { "company_id", "verification_status": "pending" } }

A company can log in immediately after email confirmation, but every restricted action (posting a job, swiping students) is blocked by the requireVerifiedCompany middleware from Milestone 7 until an admin approves the account in the Phase 11 workflow.

5.3 Verification Document Placeholder

POST /api/v1/auth/company/verification-documents *Company*

Placeholder only in Phase 2 — accepts document metadata and stores a company_requests row with status=pending. Full file upload to Supabase Storage is implemented in Phase 4/11, not here.

Request: { "document_type": "business_registration", "file_url": "" }

Response: { "data": { "request_id", "status": "pending" } }

6. Milestone 4 — Login

6.1 Student & Company Login

POST /api/v1/auth/login

Public

Shared login endpoint for students and companies. Supabase verifies email/password, then the service layer looks up the role from the users table and issues tokens.

Request: { "email", "password" }

Response: { "data": { "access_token", "refresh_token", "user": { "id", "role", ... } } }

- Unverified students (email_verified=false) receive ACCOUNT_NOT_VERIFIED instead of a token pair, with a hint to re-request an OTP
- Pending companies can still log in successfully — restriction happens at the route level, not at login, so they can view their own dashboard and verification status
- Every successful login writes a new row to sessions (device_info, ip_address) for audit and "active sessions" UI later

6.2 Admin Login

POST /api/v1/auth/admin/login

Public (admin credentials + TOTP)

Separate, non-discoverable route — never listed in public Swagger docs. Requires a valid TOTP code in addition to email/password (2FA is mandatory for admins from day one per the backend specification's security model).

Request: { "email", "password", "totp_code" }

Response: { "data": { "access_token", "refresh_token" } }

If 2FA enrollment is not yet built when this milestone starts, ship the route behind a feature flag with password-only login for the hackathon/demo path, but log a tracked TODO — do not silently ship admin auth without 2FA into anything resembling production.

7. Milestone 5 — Authentication Middleware

This is the largest and most safety-critical piece of Phase 2 — every protected route in the entire application funnels through this middleware.

7.1 authenticate.ts — Responsibilities

1. Read the Authorization header, expect Bearer <token>
2. If missing → throw UnauthorizedError('MISSING_TOKEN')
3. Verify the JWT signature and expiry using JWT_SECRET
4. If invalid/expired → UnauthorizedError('INVALID_TOKEN') / ('TOKEN_EXPIRED')
5. Load the user record (id, role, status, email_verified) – from a short-TTL Redis cache first, falling back to Postgres on a cache miss
6. If user.status = suspended/deleted → ForbiddenError('ACCOUNT_SUSPENDED')
7. Attach the resolved identity to req.user
8. Call next()

7.2 Extending the Express Request Type

Rather than casting req.user as any at every use site, declare it once via TypeScript declaration merging so the whole codebase gets type-safe access.

```
// types/express.d.ts
declare global {
  namespace Express {
    interface Request {
      user?: {
        id: string;
        role: 'STUDENT' | 'COMPANY' | 'ADMIN';
        email: string;
        verificationStatus?: string;
      };
    }
  }
}
export {};
```

7.3 Protecting a Route

```
router.get('/profile', authenticate, profileController.getMyProfile);
GET /profile → authenticate middleware → controller – the controller can now trust req.user
without re-checking anything.
```


8. Milestone 6 — Role-Based Authorization (RBAC)

8.1 Role Enum

```
enum Role {  
  STUDENT = 'STUDENT',  
  COMPANY = 'COMPANY',  
  ADMIN = 'ADMIN',  
}
```

8.2 authorize() Middleware Factory

```
export const authorize = (...allowed: Role[]) => (req, res, next) => {  
  if (!req.user) return next(new UnauthorizedError('MISSING_TOKEN'));  
  if (!allowed.includes(req.user.role)) {  
    return next(new ForbiddenError('INSUFFICIENT_ROLE'));  
  }  
  next();  
};
```

```
// Usage:  
router.get('/admin/dashboard', authenticate, authorize(Role.ADMIN), adminController.dashboard);
```

Request → authenticated → role check → controller. This middleware always runs *after* authenticate, never before, since it depends on req.user already being populated.

8.3 Test Cases

- Student token hitting an ADMIN-only route → 403 INSUFFICIENT_ROLE
- Company token hitting a STUDENT-only route → 403 INSUFFICIENT_ROLE
- No token at all hitting any protected route → 401 MISSING_TOKEN
- Valid token, correct role → 200 and controller executes

9. Milestone 7 — Pending Company Middleware

Unique to KUPC: a company can be fully authenticated (valid JWT, correct COMPANY role) and still be blocked from restricted actions because it hasn't cleared admin verification yet. This is a distinct middleware from RBAC because it checks account *state*, not role.

```
export const requireVerifiedCompany = async (req, res, next) => {  
  const company = await companiesRepo.findById(req.user.id);  
  if (company.verification_status !== 'approved') {  
    return next(new ForbiddenError('PENDING_VERIFICATION'));  
  }  
  next();  
};
```

```
router.post('/jobs', authenticate, authorize(Role.COMPANY), requireVerifiedCompany,  
jobsController.create);
```

Flow: company → pending → attempt create job → 403 with error code PENDING_VERIFICATION and a human-readable message telling the company its account is awaiting admin approval.

10. Milestone 8 — Refresh Token Rotation

10.1 Refresh Endpoint

POST /api/v1/auth/refresh

Refresh token (body or httpOnly cookie)

Exchanges a valid, unrevoked refresh token for a new access token and a new refresh token.

Request: { "refresh_token": "..." }

Response: { "data": { "access_token", "refresh_token" } }

10.2 Rotation Logic

1. Hash the incoming token and look it up in refresh_tokens
2. If not found, or revoked = true → treat as a possible theft/replay: revoke the entire session family tied to that user and return REFRESH_TOKEN_REUSE_DETECTED
3. If found and valid → mark the old token revoked = true, generate a new refresh token, store its hash, issue a new 15-minute access token
4. Return the new pair; the client discards the old refresh token immediately

Old Refresh Token → invalidate → generate new refresh token → store new token. This rotate-on-every-use pattern means a stolen refresh token is only useful once before its reuse triggers a full session-family revocation, which is the standard mitigation for refresh-token theft.

10.3 Session & Token Storage

Table	Purpose
sessions	One row per login — device_info, ip_address, created_at, expires_at; powers an "active sessions" view and lets a user revoke a specific device
refresh_tokens	token_hash (never the raw token), user_id, expires_at, revoked, created_at — raw tokens are never persisted, only their hash

11. Milestone 9 — Logout & Session Revocation

POST /api/v1/auth/logout

Authenticated

Revokes the current session's refresh token and deletes the session row. Does not merely rely on the client discarding its access token.

Request: { "refresh_token": "..." }

Response: { "success": true }

Logout → revoke refresh token → delete session → return success. Because access tokens are short-lived (15 minutes) and stateless, a logged-out access token technically remains cryptographically valid until it expires — this is an accepted tradeoff of JWTs and is mitigated by the short TTL. For immediate revocation needs (e.g. account suspension by an admin), the authenticate middleware's user-status check in Milestone 5 (step 6) closes that gap by rejecting suspended accounts on their very next request.

Optional hardening: a **POST** /auth/logout-all endpoint that revokes every refresh token and deletes every session for the user, for a "log out of all devices" security action.

12. Milestone 10 — Protected Endpoints

GET /api/v1/auth/me

Authenticated

Returns the current authenticated user's identity (id, role, email, verification status).

Response: { "data": { "id", "role", "email", "email_verified" } }

GET /api/v1/student/dashboard

Student

Placeholder dashboard route — confirms the auth + RBAC pipeline end-to-end before Phase 3 profile data exists.

GET /api/v1/admin/dashboard

Admin

Placeholder admin-only route — confirms RBAC rejects non-admins with 403.

13. Milestone 11 — Error Handling & Error Codes

Every auth failure throws a typed `AppError` subclass (`UnauthorizedError`, `ForbiddenError`, `ValidationError`) carrying a stable machine-readable code. The central error handler maps these to HTTP status and the standard response envelope — controllers never construct error JSON by hand.

Code	HTTP	Meaning
MISSING_TOKEN	401	No Authorization header present
INVALID_TOKEN	401	JWT signature invalid or malformed
TOKEN_EXPIRED	401	Access token past its expiry
INVALID_CREDENTIALS	401	Wrong email/password combination
ACCOUNT_NOT_VERIFIED	403	Student has not completed OTP verification
ACCOUNT_SUSPENDED	403	Account status is suspended or deleted
INSUFFICIENT_ROLE	403	Authenticated but wrong role for this route
PENDING_VERIFICATION	403	Company not yet approved by an admin
INVALID_OTP	400	OTP does not match
OTP_EXPIRED	400	OTP window (10 min) has passed
INVALID_EMAIL_DOMAIN	400	Student email is not a KU institutional address
REFRESH_TOKEN_REUSE_DETECTED	401	A revoked refresh token was replayed — session family revoked as a precaution
EMAIL_ALREADY_REGISTERED	409	Duplicate registration attempt

14. Milestone 12 — Request Validation (Zod)

Every auth endpoint has a dedicated Zod schema in `auth.validation.ts`, applied by a generic `validate(schema)` middleware before the controller runs. Unknown fields are stripped, not silently accepted.

Endpoint	Schema
register/student	RegisterStudentSchema — KU email regex, full_name, password policy
register/company	RegisterCompanySchema — company_name, email, password, optional website URL
verify-otp	VerifyOtpSchema — email, 6-digit numeric otp
login	LoginSchema — email, password
admin/login	AdminLoginSchema — email, password, 6-digit totp_code
refresh	RefreshSchema — refresh_token (non-empty string)
logout	LogoutSchema — refresh_token

15. Milestone 13 — Testing Matrix

Student

- Register with valid KU email → 200, OTP sent
- Register with non-KU email → 400 INVALID_EMAIL_DOMAIN
- Verify correct OTP → 200, tokens issued
- Verify incorrect OTP → 400 INVALID_OTP
- Verify expired OTP → 400 OTP_EXPIRED
- Login before verification → 403 ACCOUNT_NOT_VERIFIED
- Login after verification → 200, tokens issued
- Logout → refresh token revoked, session deleted
- Access GET /auth/me with valid token → 200

Company

- Register → 200, verification_status = pending
- Login while pending → 200 (login itself is allowed)
- Attempt POST /jobs while pending → 403 PENDING_VERIFICATION

Admin

- Login with correct credentials + TOTP → 200, tokens issued
- Access GET /admin/dashboard → 200
- Student or company token on admin route → 403 INSUFFICIENT_ROLE

JWT & Refresh

- Missing token → 401 MISSING_TOKEN
- Malformed/invalid signature → 401 INVALID_TOKEN

- Expired access token → 401 TOKEN_EXPIRED
- Valid refresh token → new access + refresh pair issued
- Expired refresh token → 401, must re-login
- Revoked (already-used) refresh token replayed → 401 REFRESH_TOKEN_REUSE_DETECTED, entire session family revoked

16. Data Model Reference

Phase 2 touches four tables directly from the full backend schema. Column sets are repeated here for convenience; the master definitions live in the Backend Specification document.

Table	Columns touched in Phase 2
users	id, email, role, email_verified, status, created_at
students	id (fk users), ku_id, full_name — other profile fields added in Phase 3
companies	id (fk users), company_name, website, verification_status, verified_at
sessions	id, user_id, device_info, ip_address, created_at, expires_at
refresh_tokens	id, user_id, token_hash, expires_at, revoked, created_at
company_requests	id, company_id, document_type, file_url, status (placeholder only)

17. Sequence Diagrams (Described)

17.1 Student Registration → Verified Login

Client → POST /register/student → Service validates + Supabase signUp → users/students rows created (unverified) → OTP emailed → Client → POST /verify-otp → OTP checked → email_verified=true → session + refresh_token created → access + refresh tokens returned → Client stores tokens → subsequent requests attach Bearer access token.

17.2 Access Token Expiry → Silent Refresh

Client makes a request with an expired access token → 401 TOKEN_EXPIRED → client intercepts this specific code → calls POST /auth/refresh with the stored refresh token → receives new access + refresh pair → original request is retried transparently → user never sees a login prompt.

17.3 Pending Company Blocked Action

Client (company, pending) → POST /jobs with valid access token → authenticate passes → authorize(COMPANY) passes → requireVerifiedCompany checks verification_status → pending → 403 PENDING_VERIFICATION → client shows "your account is awaiting approval" UI.

18. Final Project Structure & Exit Checklist

```
src/
  modules/auth/ - auth.controller.ts · auth.service.ts · auth.routes.ts · auth.validation.ts ·
auth.types.ts · auth.constants.ts · index.ts
  middleware/ - authenticate.ts · authorize.ts · requireVerifiedCompany.ts · attachUser.ts
  config/ - supabase.ts · env.ts
  database/ - users.repository.ts · sessions.repository.ts · refreshTokens.repository.ts
  types/ - express.d.ts
  utils/ - jwt.ts · auth.ts
  app.ts
  server.ts
```

Exit Checklist

- All 13 milestones complete and individually tested
- Every endpoint has a Zod schema and appears in the generated Swagger doc
- Every error path returns one of the codes from Milestone 11's table, never a raw stack trace
- Refresh token rotation verified with a manual reuse-attack test
- RBAC verified against all three roles in both directions (correct role passes, wrong role 403s)
- Pending-company gate verified against at least one restricted route (job creation)
- No service-role Supabase key referenced anywhere outside config/supabase.ts and trusted server-side services
- Feature-based module structure adopted as the pattern for all subsequent phases (profiles, resume, jobs, swipes, chat)

End of document. Once every item in the exit checklist is verified, Phase 3 — Student & Company Profiles — can begin.